POKEBALLERS

Midterm Presentation

Pulsepoint/Telegraph/Dashforge - Customizable API Monitoring Dashboard

Conor Driscoll	Backend / Database
Adrian Czerwonka	UI / Wireframes
Idrees Abdurrazzak	Frontend Lead

Project Overview — Business Case

The Problem

Developers and information-consumers today pull data from many disconnected services: uptime monitors, weather APIs, financial feeds, internal service health endpoints, and ad-hoc JSON APIs. Existing tools fall into two camps - narrow single-purpose dashboards and heavyweight technical tools requiring significant setup.

The Opportunity

This project builds in the empty middle ground: a web application where users compose grid-based boards of configurable widgets, each tied to an API or data source, viewable in a single browser tab.

Target Users

👨‍💻 Developers

Builds and maintains web projects, wants passive uptime visibility

🔍 Hobbyists

Personal user who wants a daily glance dashboard

⚙️ Data-Curious Users

Aggregates public API data from multiple sources into organized views

Technologies — Tech Stack

Frontend

- Svelte
- Skeleton UI
- Tailwind CSS
- TypeScript

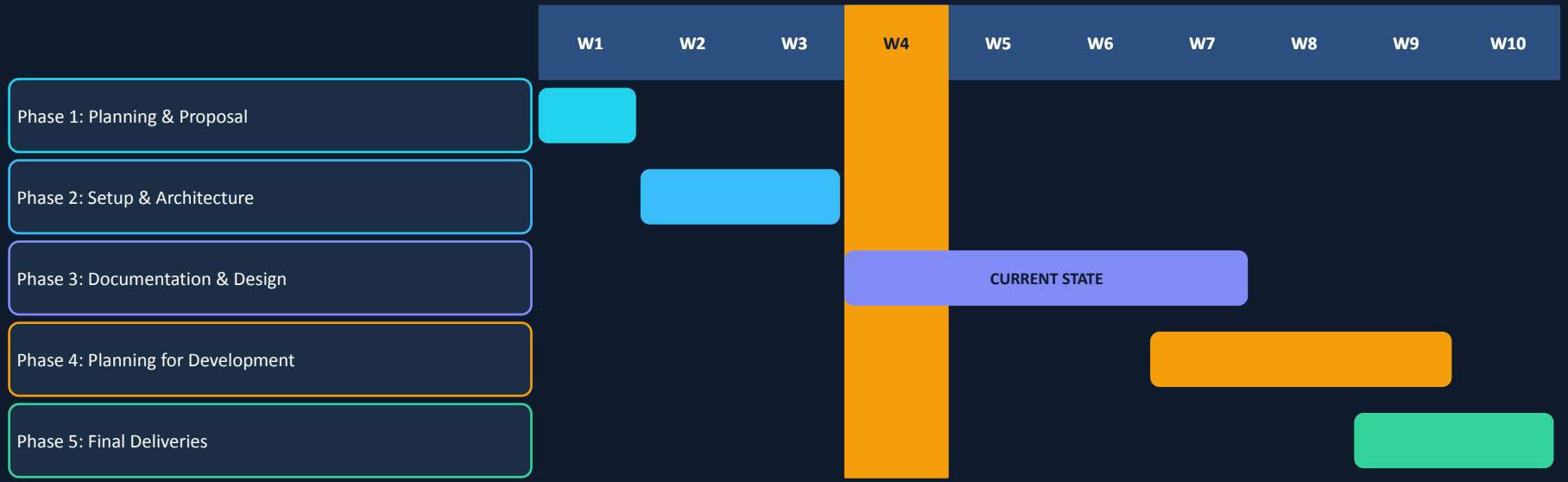
Backend

- Node.js
- Fastify
- Better-Auth
- Resend

Database

- PostgreSQL
- Drizzle ORM
- Redis
- BullMQ

Project Timeline — Gantt Chart (10 Weeks)

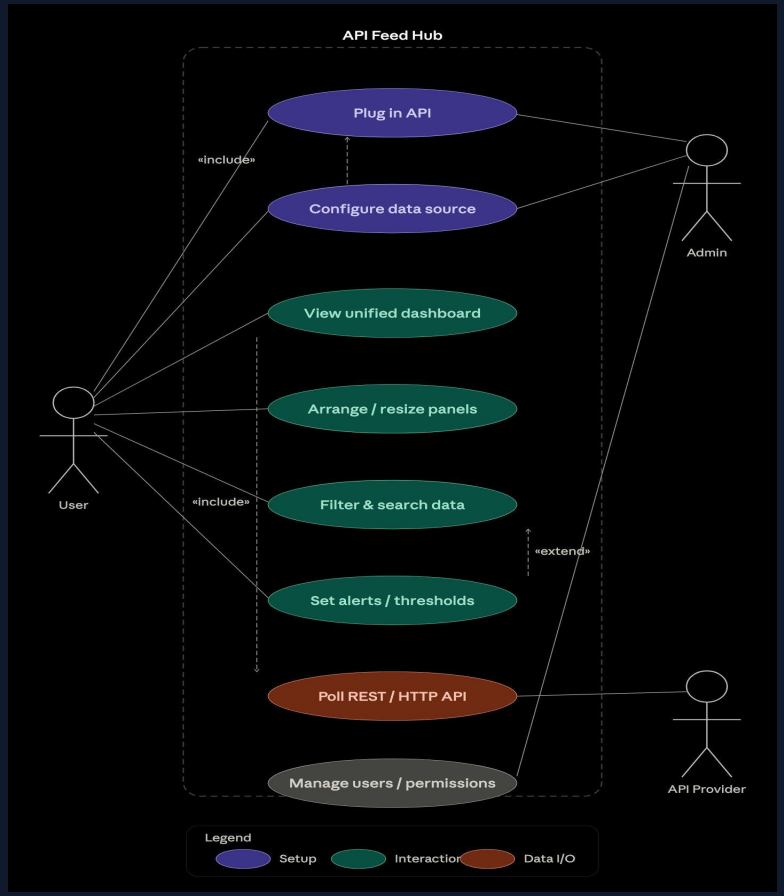


▸ **Current week: Week 5 — Phases 1 & 2 complete, Phase 3 in progress**

POKEBALLERS — Project Gantt Chart (10 Weeks) | Quarter 1: Planning & Design

Task / Deliverable	Owner	Status	Wk 1	Wk 2	Wk 3	Wk 4	Wk 5	Wk 6	Wk 7	Wk 8	Wk 9	Wk 10
Legend:			Complete		In Progress		Not Started		◆ Milestone		▶ Current Wk	
PHASE 1: PLANNING & PROPOSAL												
Define project scope & business problem	All	Complete										
Identify target users & requirements	All	Complete										
Draft & submit project proposal (PPTX)	All	Complete										
Re-submit revised proposal	All	Complete										
◆ Submit proposal milestone	All	Complete	◆									
PHASE 2: SETUP & ARCHITECTURE												
Finalize technology stack	All	Complete										
Set up GitHub repo & folder structure	Conor	Complete										
Design database schema (PostgreSQL)	Conor	Complete										
Use Case Diagram	Adrian	Complete										
Prototype / Wireframe (Figma)	Adrian	Complete										
◆ Setup milestone	All	Complete				◆						
PHASE 3: DOCUMENTATION & DESIGN												
Midterm Presentation	Idrees	In Progress										
Entity Relationship Diagram (final)	Conor	In Progress										
Start project Wiki	All	In Progress										
Finalize project / site name	All	In Progress										
Activity Diagrams	All	Not Started										
Class Diagrams	All	Not Started										
Heuristic Evaluation	All	Not Started										
◆ Documentation milestone	All	Not Started								◆		
PHASE 4: PLANNING FOR DEVELOPMENT												
Define detailed user stories & requirements	All	Not Started										
API design spec (endpoints, payloads)	Conor	Not Started										
Component architecture plan	Conor	Not Started										
Sprint planning for Q2 development	Idrees	Not Started										
Final Gantt Chart (updated for Q2)	Idrees	Not Started										
PHASE 5: FINAL DELIVERABLES												
Finalize & publish project Wiki	Adrian	Not Started										
Final Presentation (all sections)	All	Not Started										
◆ Final presentation milestone	All	Not Started										◆

Use Case Diagram — API Feed Hub



Actors

User

Plug in API, configure data source, view dashboard, arrange panels, filter data, set alerts/thresholds

Admin

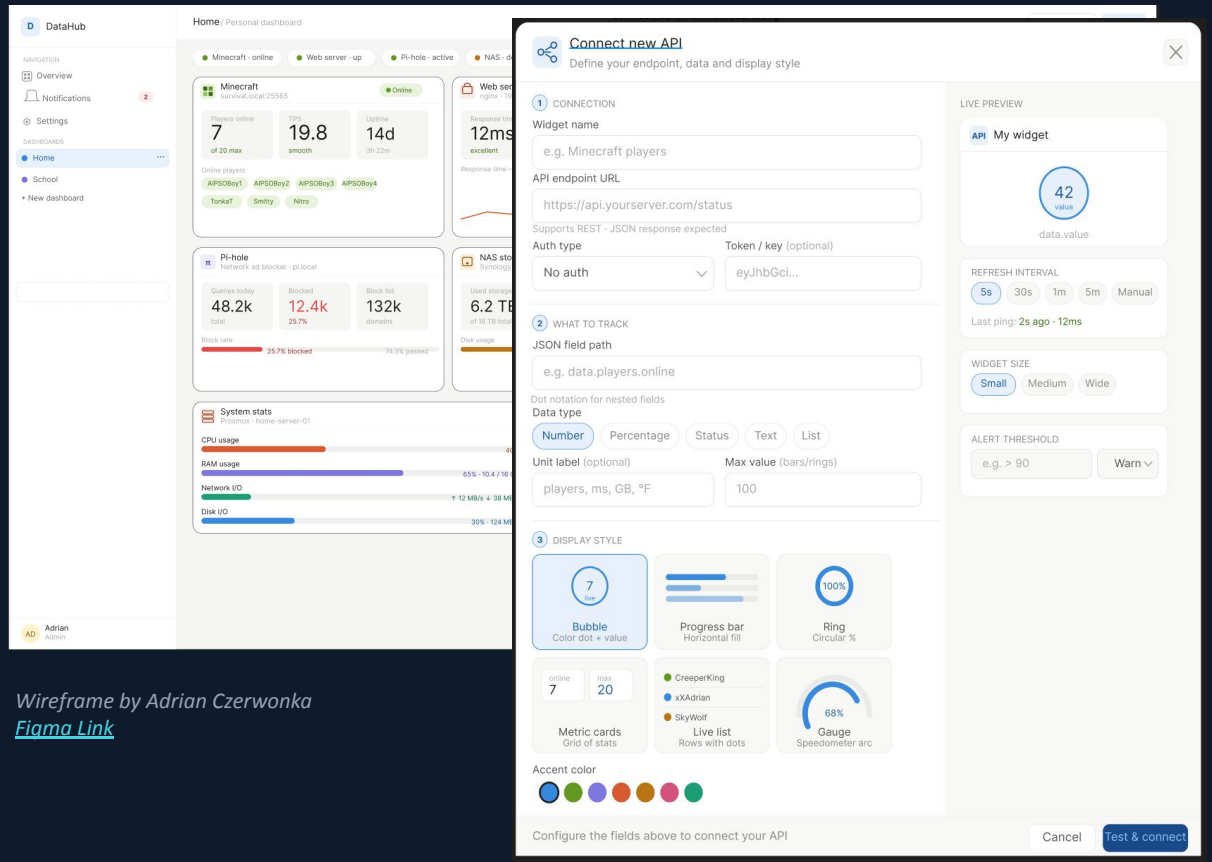
Configure data source, manage users & permissions

API Provider

Receives poll requests (Poll REST/HTTP API)

Relationships: «include» — Configure data source includes Plug in API · «extend» — Set alerts extends Filter & search

Wireframe / Prototype — Main Board



Wireframe by Adrian Czerwonka
[Figma Link]

Widgets

Tables
Display structured API response data in rows/columns

Graphs
Bar, line charts from numeric API data

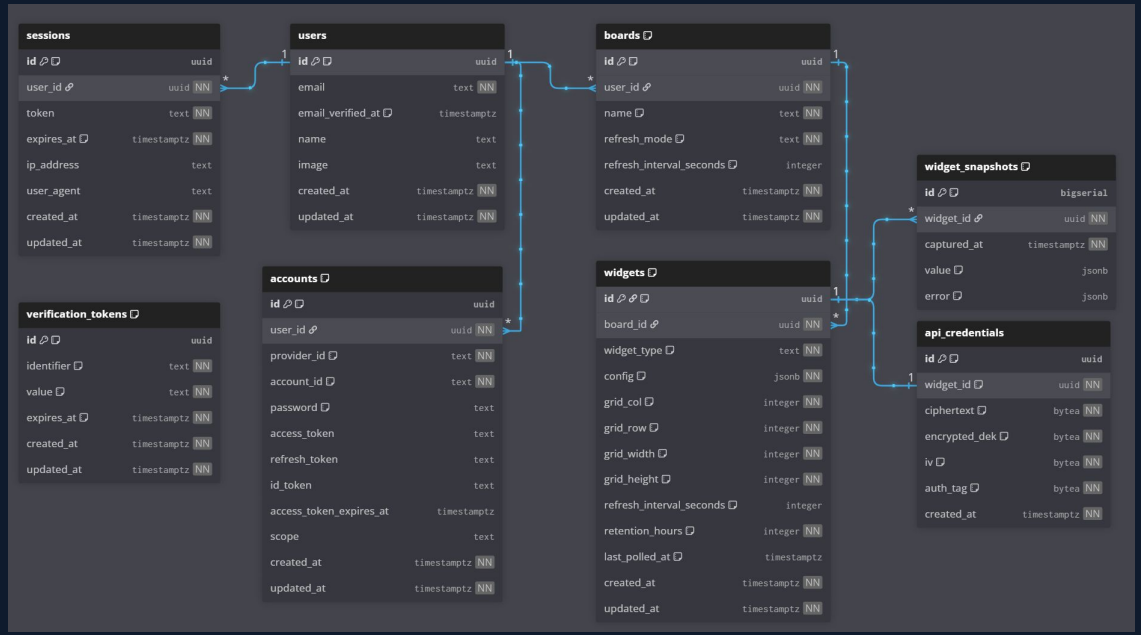
Singular Data
Single value display

Automatic Updating
Widgets can be configured to refresh while the page is active

Data snapshot retention
Data snapshots for historical displays can be stored

Notification
User can configure alerts to appear in their notifications

Entity Relationship Diagram — Database Schema



Tables

users

Auth via email or Google OAuth. Handled through Better-Auth

boards

User-owned. Has name, refresh mode, and interval settings.

widgets

Grid-positioned on a board. Stores config JSON, refresh rate.

widget_snapshots

Timestamped API response data captured per widget.

api_credentials

Encrypted API keys stored with AES (bytea + nonce).

Where We Are

Week 4 of 10 — Phases 1 & 2 Complete

✓ Completed

- Project proposal submitted & revised
- Technology stack finalized (Svelte + Skeleton UI + Node.js + PostgreSQL)
- GitHub repo & project structure set up
- Database schema designed
- Use Case Diagram completed
- Wireframe / Prototype completed (Figma)
- Gantt chart created as living document

→ Week 5 Goals

- Finalize & agree on project / site name
- Start project Wiki (tech stack, architecture, roles)
- Begin front-end dashboard UI skeleton
- Set up backend routes & PostgreSQL connections
- Schedule 60-min structured team meetings
- Review wireframes together & sign off